

MatchPoint — Documentación técnica

Backend de **MatchPoint**: reserva de canchas de básquet + torneos de eliminación directa. Documento de referencia para entender **todo el código** y **la colección de Postman**.

- **Stack**: Kotlin · Spring Boot 4 · Spring Data JPA · Spring Security (OAuth2 Resource Server) · H2 en memoria
- **Paquete base**: `com.pucetec.matchpoint`
- **Puerto**: `8787`
- **Autorización**: AWS Cognito (JWT) -> roles **MANAGER** / **PLAYER**, más validación de **propiedad**

1. Visión general

MatchPoint tiene **dos dominios** en un solo backend, unidos en **5 tablas**:

Dominio	Tablas	Qué hace
Reservas	<code>courts</code> , <code>reservations</code>	Un MANAGER publica canchas; un PLAYER las reserva
Torneos	<code>tournaments</code> , <code>teams</code> , <code>matches</code>	Un MANAGER crea torneos; un PLAYER inscribe equipos; se juega un cuadro de eliminación directa

`courts` **fusiona** el "complejo" y la "cancha" del diseño original (cada cancha lleva su sector, si tiene parqueo y quién es su dueño). Por eso son 5 tablas y no 6.

Los dos roles

Rol	Puede
MANAGER	Crear/editar sus canchas · crear torneos, arrancar el cuadro, programar y registrar marcadores de sus torneos
PLAYER	Reservar canchas, ver/cancelar sus reservas · inscribir/retirar equipos

Regla de oro: los usuarios y roles **NO** viven en la base de datos. Viven en **AWS Cognito** y llegan al backend dentro del **JWT**. El backend solo confía en el token.

2. Arquitectura en capas (¿cómo viaja una petición?)

Cada petición atraviesa las capas en este orden. **Ninguna capa hace el trabajo de otra**.

```
Cliente (Postman)
| HTTP + JWT (Bearer token)
v
[ SecurityConfig ] -- valida el token, traduce cognito:groups -> ROLE_MANAGER/ROLE_PLAYER
```

```

| (sin token -> 401 . rol incorrecto -> 403)
v
[ Controller ] -- NO tiene lógica. Recibe el body, saca el username del JWT y delega.
|
v
[ Service ] -- TODA la lógica de negocio y validaciones. Lanza excepciones propias.
| Validar "propiedad" (torneo/cancha/reserva ajena -> 403).
v
[ Repository ] -- acceso a datos (Spring Data JPA). El service nunca escribe SQL.
|
v
[ Entity ] <-> [ Mapper ] <-> [ DTO ] -- la entity vive en la BD; el mapper la
pasa a DTO; el endpoint SOLO expone DTOs.

Si algo falla -> [ GlobalExceptionHandler ] traduce la excepción al código HTTP correcto.

```

Carpeta por carpeta

config/	SecurityConfig -> reglas de seguridad (roles por endpoint + lector de JWT)
controllers/	Court, Reservation, Tournament, Team, Match, Me
services/	CourtService, ReservationService, TournamentService -> reglas de negocio
repositories/	Court, Reservation, Tournament, Team, Match -> interfaces JpaRepository
entities/	Court, Reservation, Tournament, Team (+ embebidos), Match -> tablas
dto/	un archivo por entidad (Request/Response) + MeDto
mappers/	@Component: convierten entity <-> DTO
exceptions/	BookingExceptions + TournamentExceptions + GlobalExceptionHandler
enums/	SportType (solo BASKET), TournamentStatus, MatchStatus, ReservationStatus
resources/db/	matchpoint_schema.sql -> DDL de PostgreSQL (opcional; la app corre en H2)

3. El modelo de datos (5 tablas)

```

courts --1:N--> reservations      (una cancha tiene muchas reservas)
courts --1:N--> tournaments      (una cancha puede ser sede de torneos; opcional)
tournaments --1:N--> teams        (un torneo tiene muchos equipos)
tournaments --1:N--> matches      (un torneo tiene muchos partidos)
teams <--N:M--> teams (vía 'matches': un partido enfrenta a 2 equipos)
tournaments --N:1--> teams        (champion_team_id -> equipo campeón, opcional)

```

courts (cancha = complejo + cancha fusionados)

id, name, sector, has_parking, sport_type(BASKET), floor_type, price_per_hour, active, manager_user, created_at

- manager_user = username del JWT del dueño.

reservations

id, court_id(FK), owner_user, starts_at, duration_minutes, status(CONFIRMED/CANCELLED), created_at

- owner_user = username del JWT del PLAYER que reservó.

tournaments

`id, name, sport_type, max_teams, prize(opcional), status(REGISTRATION/IN_PROGRESS/FINISHED), manager_user, court_id(FK opcional), champion_team_id(FK opcional), created_at`

- `max_teams` debe ser **potencia de 2** (2, 4, 8, 16, 32) para armar un cuadro perfecto sin "bytes".

teams — datos externos + datos internos

El equipo se parte en dos bloques embebidos (Value Objects) dentro de la misma tabla:

- **Externos (público/contacto):** `name, contact_name, contact_email, contact_phone`
- **Internos (estado del torneo):** `eliminated, matches_played, matches_won, matches_lost, points_for, points_against, current_round`
- `registered_by_user` = username del JWT del PLAYER que lo inscribió.

En el código esto son los `@Embeddable` **TeamExternalInfo** y **TeamInternalStats** dentro de `Team`.

matches (llaves del cuadro)

`id, tournament_id(FK), round_number, position_in_round, home_team_id, away_team_id, home_score, away_score, winner_team_id, status(PENDING/READY/PLAYED), scheduled_at, created_at`

4. Seguridad: cómo se protege todo

Cómo llegan los roles

1. El usuario inicia sesión en **Cognito** -> recibe un **access_token** (JWT).
2. Ese token trae el claim `cognito:groups`, p. ej. `["MANAGER"]`.
3. `SecurityConfig` lo lee y arma la authority **ROLE_MANAGER** (o **ROLE_PLAYER**), que es lo que espera `hasRole(...)`.

```
// SecurityConfig.kt (resumen)
converter.setJwtGrantedAuthoritiesConverter { jwt ->
    val groups = jwt.getClaimAsStringList("cognito:groups") ?: emptyList()
    groups.map { SimpleGrantedAuthority("ROLE_${it.uppercase()}") }
}
```

Dos capas de 403 (¡importante para la sustentación!)

Código	Significa	¿Quién lo pone?
401	No sé quién eres (sin token o token inválido)	Spring Security
403 por rol	Sé quién eres, pero tu rol no puede hacer esta acción	Spring Security (<code>hasRole</code>)

403 por propiedad	Eres MANAGER... pero ese torneo/cancha no es tuyo	TÚ, en el service
404	El recurso no existe	TÚ, en el service
409	El estado no permite la operación (cupó lleno, cancha ocupada, partido ya jugado...)	TÚ, en el service

El 403 por propiedad se hace con un patrón repetido en cada service:

```
private fun findTournamentOwnedBy(id: Long, managerUser: String): Tournament {
    val t = findTournament(id) // 404 si no existe
    if (t.managerUser != managerUser) // no es tuyo
        throw NotYourTournamentException("El torneo $id no es tuyo") // -> 403
    return t
}
```

(Equivalentes: `CourtService.findCourtOwnedBy`, `ReservationService.findReservationOwnedBy`.)

El "yo" sale del token, nunca del body

Los DTOs de request **no tienen** campos de dueño (`managerUser`, `ownerUser`, `registeredByUser`). El controller lo saca del JWT y se lo pasa al service:

```
fun create(@RequestBody request: CreateCourtRequest, @AuthenticationPrincipal jwt: Jwt) =
    courtService.createCourt(request, jwt.username()) // username = claim del token

private fun Jwt.username(): String = getClaimAsString("username")
```

Así **nadie puede suplantar a otro**: aunque metas `"managerUser": "otro"` en el body, Jackson lo ignora (no está en el DTO) y el dueño siempre es quien manda el token.

5. Explicación capa por capa

5.1 Entities (las tablas)

Clases `@Entity` con el idiom del proyecto: `id: Long = 0` al final, campos mutables `var`, `createdAt` por defecto. Las relaciones son `@ManyToOne`. `Team` usa `@Embedded` para separar datos internos/externos.

5.2 Repositories (acceso a datos)

Interfaces que extienden `JpaRepository`. Spring genera la implementación sola. Solo declaramos consultas por convención de nombre, p. ej.:

- `ReservationRepository.findByOwnerUserOrderByStartsAtDesc(user)` -> reservas de un usuario.
- `TeamRepository.existsByTournamentIdAndExternalInfoName(id, name)` -> evita nombres de equipo repetidos.
- `MatchRepository.findByTournamentIdAndRoundNumberAndPositionInRound(...)` -> ubica una llave del cuadro.

5.3 DTOs (lo que entra y sale)

- **Request:** lo que acepta el endpoint (sin campos de dueño).
- **Response:** lo único que devuelve (nunca la entity). Un archivo por entidad: `CourtDto`, `ReservationDto`, `TournamentDto`, `TeamDto`, `MatchDto`, `MeDto`.

5.4 Mappers (@Component)

Convierten entity <-> DTO. Son **componentes inyectados** en los services. Ej:

```
@Component
class CourtMapper {
    fun toEntity(req: CreateCourtRequest, managerUser: String): Court { ... } // dueño por p
    fun toResponse(court: Court): CourtResponse { ... }
    fun toResponseList(courts: List<Court>) = courts.map { toResponse(it) }
}
```

5.5 Services (el cerebro)

Aquí vive **toda** la lógica. Ejemplos de reglas:

- **Canchas:** no se crea con precio ≤ 0 ; solo el dueño edita (403); disponibilidad = activa + sin solapamiento.
- **Reservas:** no se reserva una cancha inactiva ni una franja ya ocupada (409); solo el dueño cancela.
- **Torneos:** `maxTeams` debe ser potencia de 2 (si no, 400); solo el dueño arranca/registra marcador.

5.6 Controllers (la puerta)

Delgados: mapean la URL, validan el body de forma básica, sacan el `username` del JWT y llaman al service. No arman respuestas de error ni contienen `if` de negocio.

5.7 Exceptions + GlobalExceptionHandler

Las excepciones propias expresan reglas de negocio. El `@RestControllerAdvice` las traduce a HTTP:

```
@ExceptionHandler(NotYourTournamentException::class, NotYourCourtException::class, NotYourRes
@ResponseStatus(HttpStatus.FORBIDDEN)
fun handleForbidden(ex: RuntimeException) = mapOf("error" to ex.message)
```

6. La lógica clave

6.1 Motor del cuadro de eliminación directa (torneos)

Cuando el MANAGER da `POST /tournaments/{id}/start`:

1. Se valida que haya **exactamente** `maxTeams` equipos (cupó lleno).
2. **Ronda 1:** se emparejan los equipos en `maxTeams/2` partidos (estado `READY`).
3. **Rondas siguientes:** se crean partidos **vacíos** (`PENDING`) que esperan a los ganadores.
4. `totalRondas = log2(maxTeams)` (para 4 equipos -> 2 rondas: semifinal y final).

Cuando el MANAGER da **PATCH** `/matches/{id}/score`:

1. Se registra el marcador (no se permiten empates -> 400).
2. El perdedor queda `eliminated = true`; se actualizan estadísticas de ambos equipos.
3. El **ganador avanza**: el ganador de (ronda `r`, posición `p`) pasa al partido (`r+1`, `p/2`), al lado **local** si `p` es par o **visitante** si `p` es impar. Cuando la llave siguiente tiene sus dos equipos, pasa a **READY**.
4. Si el partido era la **final**, se declara **campeón** y el torneo pasa a **FINISHED**.

```
4 equipos:
Ronda 1 (Semifinales)          Ronda 2 (Final)
pos0: A vs B -ganador+
                                +--> pos0: (ganador SF1) vs (ganador SF2) -> CAMPEÓN
pos1: C vs D -ganador+
```

6.2 Reservas sin choques de horario

Al reservar, el service busca las reservas **CONFIRMED** de esa cancha y rechaza si hay **solapamiento**:

```
dos franjas [inicioA, finA) y [inicioB, finB) se solapan <=> inicioA < finB Y inicioB < f
```

Si se solapa -> **409 CourtNotAvailableException**. Cancelar una reserva la deja **CANCELLED** (libera la franja).

7. Los 22 endpoints (>=4 por tabla)

Sin token -> **401** · rol equivocado -> **403** · recurso ajeno -> **403** (service).

Canchas (**courts**, 5)

Método	Endpoint	Acceso
GET	<code>/courts?sector=&sport=</code>	Público
GET	<code>/courts/available?sector=&sport=&startsAt=&durationMinutes=</code>	Público
GET	<code>/courts/{id}</code>	Público
POST	<code>/courts</code>	MANAGER
PATCH	<code>/courts/{id}</code>	MANAGER dueño

Reservas (**reservations**, 4)

Método	Endpoint	Acceso
POST	<code>/reservations</code>	PLAYER
GET	<code>/reservations/me</code>	PLAYER
GET	<code>/reservations/{id}</code>	PLAYER dueño

DELETE	/reservations/{id}	PLAYER dueño
--------	--------------------	--------------

Torneos (tournaments, 4)

Método	Endpoint	Acceso
GET	/tournaments	Público
GET	/tournaments/{id} (progreso + cuadro)	Público
POST	/tournaments	MANAGER
POST	/tournaments/{id}/start	MANAGER dueño

Equipos (teams, 4)

Método	Endpoint	Acceso
GET	/tournaments/{id}/teams	Público
GET	/tournaments/{id}/teams/{teamId}	Público
POST	/tournaments/{id}/teams	PLAYER
DELETE	/tournaments/{id}/teams/{teamId} (solo en inscripción)	PLAYER dueño

Partidos (matches, 4)

Método	Endpoint	Acceso
GET	/tournaments/{id}/matches	Público
GET	/matches/{matchId}	Público
PATCH	/matches/{matchId}/schedule	MANAGER dueño
PATCH	/matches/{matchId}/score	MANAGER dueño

Identidad

Método	Endpoint	Acceso
GET	/me	Autenticado (devuelve username, sub, groups del token)

8. La colección de Postman

Archivo: `matchpoint.postman_collection.json`. Está pensada para **probar de corrido**.

Variables (Colección -> Variables)

Variable	Para qué
<code>base_url</code>	<code>http://localhost:8787</code>
<code>token_manager</code> / <code>token_player</code>	tokens de Cognito (los pegas tú)
<code>court_id</code> , <code>tournament_id</code> , <code>reservation_id</code> , <code>sf1_id</code> , <code>sf2_id</code> , <code>final_id</code>	se llenan solos con scripts

Los scripts que encadenan todo

Algunos requests tienen un **Test script** que guarda el `id` de la respuesta en una variable, para que el siguiente request lo use sin copiar nada a mano. Ejemplos:

- `POST /courts` -> guarda `court_id`
- `POST /tournaments` -> guarda `tournament_id`
- `POST /reservations` -> guarda `reservation_id`
- `POST /tournaments/{id}/start` -> lee el cuadro de la respuesta y guarda `sf1_id`, `sf2_id`, `final_id`

```
// Ejemplo (POST /courts): guarda el id devuelto
pm.test('201 Created', () => pm.response.to.have.status(201));
pm.collectionVariables.set('court_id', pm.response.json().id);
```

Además, casi todos verifican el **status** (verde/rojo en la pestaña *Test Results*).

Las 5 carpetas

Carpeta	Qué prueba
1. MANAGER · Canchas + crear torneo	crea cancha, la lista/consulta/edita, crea el torneo
2. PLAYER · Equipos + reservas	inscribe 4 equipos, reserva una cancha, ve sus reservas
3. MANAGER · Arrancar, programar y jugar	genera el cuadro, programa, registra marcadores -> campeón
4. Seguridad (401 / 403)	sin token -> 401; rol equivocado -> 403 (6 casos)
5. Otros	<code>/me</code> (manager/player), cancelar reserva, retirar equipo

Cómo correrla

1. **Importa** el archivo en Postman.
2. En **Variables**, pega `token_manager` y `token_player` (ver sección 9).
3. Levanta el backend: `./gradlew bootRun`.
4. Corre en orden las carpetas **1 -> 2 -> 3** (o usa el **Collection Runner**). Los ids se guardan solos.
5. La carpeta 4 demuestra los 401/403 para la evidencia.

9. Autenticación con Cognito (resumen)

1. En **AWS Cognito** existe un **User Pool** con los grupos **MANAGER** y **PLAYER** y dos usuarios (**manager1** -> **MANAGER**, **player1** -> **PLAYER**).
2. En **application.yaml**, el **issuer-uri** apunta a ese pool:

```
issuer-uri: https://cognito-idp.us-east-1.amazonaws.com/us-east-1_JDqEph0S3
```
3. Para obtener un token, en Postman se usa **OAuth 2.0 -> Authorization Code (With PKCE)** con el dominio del pool (**/oauth2/authorize** y **/oauth2/token**), el Client ID (y secret si aplica) y scope **openid**.
4. El **access_token** resultante (un JWT que empieza con **eyJ...**) se pega en **token_manager** / **token_player**.
5. Verificación rápida: **GET /me** -> debe devolver el **username** y **groups** correctos.

El backend **no valida contraseñas ni guarda usuarios**: solo verifica la firma y el emisor del JWT (contra el **issuer-uri**) y lee sus claims. Por eso, si cambias de pool, hay que **reiniciar** el backend.

10. Cómo correr el proyecto

```
cd /Users/macnoe/josue/match
./gradlew bootRun      # arranca en http://localhost:8787
./gradlew test         # 56 pruebas (unitarias + de seguridad), sin necesidad de AWS
```

- Consola H2: **http://localhost:8787/h2-console** (JDBC **jdbc:h2:mem:matchpointdb, admin/admin**).
- La base es **en memoria**: cada reinicio empieza limpia.
- Para desplegar en **PostgreSQL**: ejecuta **src/main/resources/db/matchpoint_schema.sql**, cambia el **datasource** en **application.yaml** y pon **ddl-auto: validate**.

Las pruebas automáticas (56)

Clase	Qué cubre
TournamentServiceTest (20)	cupo, empates, propiedad, avance de ganador, campeón, retirar equipo, programar
CourtServiceTest (5)	crear, editar (propiedad), disponibilidad con solapamiento
ReservationServiceTest (7)	reservar, cancha inactiva/ocupada, cancelar (propiedad)
TournamentSecurityTest (13)	401/403 por rol en torneos, equipos y partidos
BookingSecurityTest (9)	401/403 por rol en canchas y reservas
MeControllerTest (2)	/me lee el rol del token

Glosario rápido

- **JWT / access_token**: credencial firmada que Cognito da al usuario; trae `username` y `cognito:groups`.
- **Resource Server**: rol de Spring Security que **valida** el JWT (no lo emite).
- **DTO**: objeto de entrada/salida de la API (nunca se exponen las entidades).
- **Mapper**: convierte entidad <-> DTO.
- **Propiedad (ownership)**: que un recurso pertenezca al usuario del token; si no, `403`.
- **Cuadro / bracket**: estructura de partidos de eliminación directa.